SIMATS ENGINEERING

MOBILE APPLICATION DEVELOPMENT ENVIRONMENT

ASSIGNMENT 4 – PART I & PART II (CO4)

MOBILE SYSTEMS ENGINEERING: ARCHITECTURE, RUNTIME, SECURITY AND POWER MANAGEMENT

Submitted by: **B Chaithanya Prakash Reddy**

Register Number: **192425255**

Department: **AI&ML**

Course: **Mobile Application Development Environment**

IMPORTANT NOTE ABOUT THE ASSIGNMENT

The supplied PDF contains two separate individual assignments. Part I is a design and analysis assignment and explicitly does not require an IDE, emulator or SDK. Part II requires implementation, commands, screenshots, logs, profiler traces and measured results. This report provides the written design, analysis, pseudocode, test plans and expected interpretations for both parts. Device-specific measurements, screenshots and actual command outputs are intentionally identified as evidence to be collected; they are not fabricated.

For Part II, replace every bracketed evidence field with output collected from your own Android device or emulator before submission. The assignment requires source code, test input, expected behaviour, actual output and intermediate system evidence—not only a final result.

PART I – ARCHITECTURAL ANALYSIS AND DESIGN

TASK 1 – ARCHITECTURE REVIEW

1.1 Diagnose the draft description

• Missing layers such as device drivers and hardware abstraction — Why it is insufficient: An engineer cannot determine how hardware is controlled or where device-specific operations are isolated.

• Kernel responsibilities are vague — Why it is insufficient: The note does not identify process scheduling, memory management, networking, device drivers, security boundaries or system-call interfaces.

• Runtime is not defined — Why it is insufficient: It does not say whether the runtime loads bytecode, native code or both, or whether compilation occurs at install time, load time or execution time.

• Permissions are described as a general promise — Why it is insufficient: It does not identify who requests and grants permission, when consent occurs, or which OS component enforces access.

• Power management has no trigger or mechanism — Why it is insufficient: There is no description of idle detection, CPU frequency scaling, suspend, wake locks or background-work restrictions.

• Application-to-OS boundary is missing — Why it is insufficient: The APIs and IPC path through which an app requests system services are not identified.

• Native libraries and framework services are omitted — Why it is insufficient: The note does not show graphics, media, database, networking and higher-level application services.

1.2 Repaired architecture – Android

Selected platform: Android. The following layered specification identifies each layer, its responsibility, the interface exposed upward and its dependency below.

Layer	Responsibility	Exposes to layer above	Depends on
Applications	User-facing apps and application components such as Activity, Service and BroadcastReceiver.	Application features and calls to Android framework APIs.	Application Framework and Android Runtime.
Application Framework	Provides Activity Manager, Window Manager, Package Manager, Notification Manager, Location Manager and other system services.	Stable Java/Kotlin APIs and system-service interfaces.	Android Runtime, Binder IPC and native system services.
Android Runtime (ART)	Loads and executes DEX application code; performs verification, interpretation,	Runtime execution, class loading, memory management and JNI access.	Linux kernel process, memory and thread services; native libraries.

	JIT/AOT compilation and garbage collection.		
Native Libraries and HAL	Native libraries provide graphics, media, database, SSL and other functions. HAL gives standard interfaces to hardware-specific implementations.	Native APIs and hardware service interfaces.	Device drivers and kernel interfaces.
Linux Kernel	Manages processes, threads, virtual memory, networking, device drivers, security enforcement support and power-related mechanisms.	System calls, device interfaces, scheduling and memory services.	Physical hardware and firmware.

Security enforcement location: Android application sandboxing is based on per-app Linux UID isolation and kernel enforcement; SELinux adds mandatory access control. Runtime permissions are checked through framework/package services and enforced at protected APIs and lower-level resources. Power management: Android system services coordinate Doze, App Standby and job scheduling; kernel mechanisms handle CPU scheduling, suspend and wake locks. Applications request access through framework APIs and Binder IPC rather than directly controlling protected kernel resources.

1.3 Design defence (under 100 words)

This specification is architectural because it names each layer, assigns concrete responsibilities, identifies the interface exposed upward and states the dependency below. It locates ART, the Linux kernel, HAL, native libraries, security enforcement and power management. It also explains the application-to-system-service path through framework APIs and Binder IPC. These details allow another engineer to identify component boundaries, trace a request and evaluate whether a responsibility is implemented at the correct layer. It is therefore more precise than a general statement that the OS is secure or saves battery.

TASK 2 – ANDROID, iOS AND WINDOWS COMPARISON

2.1 Requirement model

- Input — Definition: Architecture and runtime properties of Android, iOS and Windows mobile platforms.
- Output — Definition: A platform comparison and a justified decision about background message delivery.
- Objective — Definition: Deliver incoming messages reliably while protecting data and limiting battery use.
- Constraints — Definition: Background execution limits, security, battery, responsiveness, network availability and OS lifecycle decisions.
- Decision — Definition: Select platform-specific implementation and notification strategy.
- Required information — Definition: Kernel family, system APIs, runtime, application framework, process isolation, IPC and push-notification services.

2.2 Hypothesis

The platform's background-execution and process-lifecycle policy is expected to affect message delivery most because it determines whether an application process can remain active or must rely on an OS-managed push service when the screen is off.

2.3 Comparison matrix

• Kernel family — Android: Linux kernel with Android-specific changes and device drivers. — iOS: Darwin foundation, including XNU kernel; user-space frameworks provide application services. — Windows: Windows NT-family kernel architecture.

• Kernel interface — Android: Linux system calls and Android-specific interfaces; apps normally access services through framework APIs. — iOS: Darwin/POSIX and system frameworks; app code normally uses public SDK APIs. — Windows: Windows system interfaces and Win32/WinRT abstractions.

• Runtime — Android: ART executes DEX code; uses interpretation, JIT and profile-guided AOT. — iOS: Objective-C runtime supports dynamic dispatch and Objective-C object/runtime services; Swift code is compiled to native code. — Windows: WinRT is an API/runtime programming model; .NET applications may use the CLR, while native apps use native execution.

• Application framework — Android: Android SDK framework, Activity/Service components, Jetpack libraries. — iOS: Cocoa Touch/UIKit and system frameworks; app lifecycle managed by UIApplication/scene APIs. — Windows: WinRT/UWP application APIs and Windows application lifecycle model.

• Isolation — Android: Per-application UID sandbox, permissions and SELinux policy. — iOS: Per-application sandbox, entitlements and code-signing restrictions. — Windows: AppContainer and capability-based restrictions for packaged apps; desktop app model differs.

• IPC — Android: Binder is the primary Android system IPC mechanism; intents and services use framework mechanisms. — iOS: XPC and Mach ports are used for system and process communication. — Windows: COM/RPC and Windows messaging mechanisms; WinRT provides application-facing APIs.

• Messaging consequence — Android: Use high-priority push through the platform messaging service where permitted; background services are restricted by OS policy. — iOS: Use APNs and permitted background modes; continuous background execution is restricted. — Windows: Use supported push/background task facilities; execution depends on app model and OS policy.

2.4 Desk-check: incoming message while screen is off

Android trace

1. Network event arrives at the device modem/Wi-Fi interface.
2. Kernel network stack and driver receive the packet and deliver it to the appropriate system networking path.
3. Google/Android push infrastructure and system services process the push notification.
4. The OS may start or wake the app's permitted receiver/service callback; delivery is subject to background restrictions and priority policy.
5. ART loads or resumes the application code and invokes the callback.
6. The app validates the message, stores it securely and posts a notification if allowed.

iOS trace

7. The APNs notification reaches the device through the network interface.
8. Kernel and networking services deliver it to the system notification infrastructure.
9. iOS notification services evaluate the notification and app execution policy.
10. The system displays an alert, or grants limited background execution for an eligible notification type.
11. The application runtime invokes the permitted notification/background callback if the app is eligible to run.
12. The app processes permitted content and updates local state.

2.5 Changed condition: app terminated due to memory pressure

Prediction: delivery differs because an application process that has been terminated cannot be assumed to receive a direct in-process callback. The responsible architectural element is the OS process manager and push-notification subsystem. On Android, a qualifying push may cause the system to start the app's receiver/service subject to message priority and background restrictions. On iOS, APNs can display an alert without launching the app; silent/background notifications are best-effort and may be delayed or omitted. The prediction should be checked using process logs and notification delivery records on the selected OS versions.

Trace the relevant path: push service receives message → OS notification subsystem checks app state and policy → alert is displayed or eligible background launch is scheduled → app process starts only if permitted → callback runs. The final observed result must be recorded from the test device.

TASK 3 – RUNTIME ENVIRONMENT ANALYSIS

3.1 Execution model

Platform	Delivered form	Loader	When executable instructions are available
Android (Dalvik-era model / ART today)	DEX bytecode in APK; native libraries may also be included.	Package manager and runtime class loader/ART.	DEX can be interpreted/JIT-compiled at runtime and AOT-compiled based on profiles/install compilation; native libraries are already machine code.
iOS	Native executable code in signed application bundle; Objective-C metadata/runtime support is included.	dyld and system launch services.	Native machine instructions are produced during build; loader maps and links the executable at launch.
Windows / WinRT	Native binaries or managed .NET assemblies depending on application technology.	Windows loader and relevant runtime.	Native code is executable after loading; managed code is JIT-compiled by CLR as needed, with AOT options in some deployment models.

3.2 Android launch sequence

13. User taps the app icon; Launcher sends an activity launch request through system services.
14. ActivityTaskManager/ActivityManager checks the target component, process state and launch permissions.
15. Zygote forks a new application process if one is not already available; the new process receives its app UID and security context.
16. ART initialises the runtime, class loader and application process environment.
17. The framework creates the Application object and invokes its lifecycle callback.
18. The target Activity is instantiated and its lifecycle methods are called in order, including onCreate, onStart and onResume as appropriate.
19. The app inflates or composes its UI, measures and lays out views, and submits drawing work.
20. Window/compositor and display pipeline render the first visible frame.

Each step depends on the preceding process, runtime and framework initialisation. The framework cannot call application lifecycle methods until the process and runtime are ready; rendering cannot complete until the activity has created its content and the window/display pipeline is available.

3.3 Runtime strategy comparison

• Execution strategy — Android ART: DEX verification, interpretation, JIT and profile-guided AOT compilation. — iOS Objective-C / Swift: Swift and most app code compiled to native machine code; Objective-C runtime supports dynamic dispatch and metadata. — Windows runtime: Native code runs directly; managed .NET code uses CLR/JIT or available AOT paths.

• Memory management — Android ART: Tracing garbage collection for managed heap; native memory has separate ownership rules. — iOS Objective-C / Swift: Swift uses ARC reference counting; Objective-C commonly uses ARC with runtime support. — Windows runtime: Managed .NET uses garbage collection; native applications use explicit/RAII-style ownership patterns.

• Native access — Android ART: JNI and Android NDK connect managed code to native libraries and system interfaces. — iOS Objective-C / Swift: Objective-C/Swift can call system frameworks and C/C++ interfaces through supported language bridges. — Windows runtime: WinRT projections and native APIs expose OS functionality.

• Start-up and footprint — Android ART: Compilation profiles can improve hot-code performance; runtime and managed heap add overhead. — iOS Objective-C / Swift: Native code can start efficiently, but framework and dynamic-linking costs remain. — Windows runtime: Varies by native or managed application model; CLR startup and JIT can add cost.

• Portability — Android ART: Java/Kotlin source and DEX runtime support aid portability across Android devices. — iOS Objective-C / Swift: Native APIs and Apple toolchain target Apple platforms; not directly portable to Android. — Windows runtime: WinRT APIs are tied to Windows; .NET code can be portable where compatible runtimes exist.

For a low-memory device, no runtime is universally best. A small native application with a limited object graph can avoid a large managed heap, but manual memory management increases defect risk. ART offers managed memory and broad device support but may require heap and GC tuning. The correct decision depends on measured resident memory, allocation rate, pause time and workload on the target device.

3.4 Cold start versus warm start

• Process creation — Cold start: Usually required if no process exists. — Warm start: Skipped if the process remains alive.

• Runtime initialisation — Cold start: Required. — Warm start: Usually already completed.

• Class/resource loading — Cold start: Required for initial use; some caches are empty. — Warm start: Often partly cached; additional loading may still occur.

• Application/activity creation — Cold start: Required. — Warm start: Activity may be recreated or resumed depending on state.

• First frame — Cold start: Must be produced from initial state. — Warm start: May appear faster if process and resources are retained.

3.5 Dependency analysis

The kernel must first provide a process, address space, threads and memory. The runtime then initialises class loading, garbage collection and execution support. Framework services can only invoke application callbacks after those dependencies are ready. Finally, the UI framework can create a window and submit rendering work. Reversing this order would leave callbacks without a process/runtime and rendering without a valid window or application state.

TASK 4 – SECURITY FORENSICS

4.1 Defect analysis

• Request every permission at installation for future use. — Mechanism bypassed: Least-privilege and runtime permission model. — Risk: Users cannot make informed, purpose-specific choices; excessive access increases impact of compromise.

• Store login token in a file readable by other apps. — Mechanism bypassed: Application sandboxing and secure storage. — Risk: Another app or a compromised device may read the token and impersonate the user.

• Accept updates from any source without producer verification. — Mechanism bypassed: Code signing and package integrity. — Risk: An attacker can install a modified or malicious update.

• Use unencrypted server communication. — Mechanism bypassed: TLS/transport protection. — Risk: An attacker on the network can read or modify credentials and messages.

4.2 Attack scenario

A user installs SecureNotes and grants broad permissions. The app stores its login token in a shared readable location. A malicious application with access to that location reads the token. Confidentiality is first lost at the moment the token is read. The attacker then uses the token to access the server. Separately, a network attacker can intercept unencrypted traffic and alter a message; integrity is first lost when the client accepts the modified response as authentic.

4.3 Minimal repair – Android

- Request each dangerous permission only when the related feature is used, and explain the purpose before requesting it.
- Store authentication secrets using Android Keystore-backed encryption; keep app files private in internal storage.
- Accept updates only through a trusted distribution channel and verify package signing identity and update signature continuity.
- Require HTTPS with TLS certificate validation; do not allow cleartext traffic for sensitive endpoints.

These repairs use the Android framework permission model, per-app UID sandbox and SELinux enforcement, Keystore, package-signing verification and TLS networking stack. They correct the identified decisions without redesigning the whole application.

4.4 Regression check

Legitimate behaviour: reading contacts after the user explicitly agrees. The app requests READ_CONTACTS only when the user selects the contact-import feature. If granted, the Contacts Provider returns permitted contact data through the framework. If denied, the app continues to support manual entry. Token encryption, signing verification and HTTPS do not block the permitted contact-import flow.

TASK 5 – POWER MANAGEMENT REASONING

5.1 Power-state policy

• Active — Hardware/kernel behaviour: CPU cores run as needed; display and radios may be active; scheduler runs foreground work. — Entry condition: User interaction, active tracking session or urgent work. — Exit condition: Screen idle and no immediate work, or explicit suspend request. — Permitted work: Foreground UI, sensor sampling at configured rate, short background tasks.

• Idle — Hardware/kernel behaviour: Display off; CPU frequency reduced; background work deferred; radios may enter low-power states. — Entry condition: No interaction for a configured period and no active high-priority task. — Exit condition: User activity, scheduled job, motion event or push event. — Permitted work: Alarm/job windows, batched sensors and limited maintenance.

• Suspended — Hardware/kernel behaviour: CPU mostly suspended; only permitted wake sources remain active. — Entry condition: No runnable foreground work and suspend conditions satisfied. — Exit condition: Hardware interrupt, permitted push/wake event, alarm or user input. — Permitted work: Minimal system wake handling; app runs only after OS grants execution.

5.2 Urgent push while suspended

A policy that permanently disables all wake sources or blocks all background execution is insufficient because urgent messages would not be processed promptly. Use OS-managed push delivery to wake the device briefly, then process the notification within the permitted execution window. Use a partial wake lock only for a short, bounded operation that genuinely requires CPU time. The application must stop holding a permanent wake lock and stop continuously polling sensors/network while idle.

5.3 Battery saver at 15%

• Kernel/hardware — Change at 15% battery saver: Prefer efficiency cores and lower CPU frequency/voltage; reduce unnecessary wake-ups and display/radio activity where policy allows.

• Runtime/system services — Change at 15% battery saver: Defer non-urgent jobs, reduce background execution windows and prioritise essential work.

• Application — Change at 15% battery saver: Reduce sensor sampling frequency, batch records, delay non-urgent uploads and avoid repeated network polling.

User-visible trade-off: location updates may become less frequent and non-urgent synchronisation may be delayed. This is acceptable because it extends the remaining operating time while preserving urgent notifications and critical tracking.

TASK 6 – RURALCARE FIELD-DATA COLLECTION DESIGN

Stage 1: Problem model

• Input — Definition: Patient data, device class, battery state, network state, user identity and shift duration.

• Output — Definition: Secure local records and synchronised server records without loss or duplication.

• Objective — Definition: Complete a ten-hour shift, protect patient data if a device is lost and synchronise safely after connectivity returns.

• Constraints — Definition: Class A Android, Class B iPhone, Class C J2ME/WML feature phone; hours offline; limited battery/storage; privacy and no duplicate records.

• Decision — Definition: Use a platform-specific application for modern smartphones and a restricted legacy interface for feature phones.

Stage 2: Platform strategy

Use native Android for Class A and native iOS for Class B because each platform provides its own secure storage, background scheduling, permission and power-management APIs. For Class C, provide a lightweight J2ME client where supported, or a WML/browser workflow where only a WML browser exists. The legacy

client stores the minimum possible data and synchronises through authenticated server sessions when connectivity is available. Restrict sensitive offline data on Class C if secure storage or encryption cannot be guaranteed.

Stage 3: Constraint table

• A – Android — Runtime: ART; Android framework. — Security: App sandbox, runtime permissions, Keystore, TLS. — Power controls: Doze, WorkManager, JobScheduler, foreground service when justified. — Offline storage: Encrypted database/files; Keystore-protected key. — Connectivity: Wi-Fi, cellular and intermittent network.

• B – iPhone — Runtime: Native iOS runtime; Swift/Objective-C frameworks. — Security: Sandbox, entitlements, Keychain, Data Protection, TLS. — Power controls: Background Tasks, permitted background modes, system-managed power. — Offline storage: Protected local database/files; Keychain-managed secrets. — Connectivity: Wi-Fi and cellular; background transfer subject to OS policy.

• C – J2ME/WML — Runtime: J2ME MIDP/CLDC runtime or WML browser. — Security: Limited device-dependent controls; server-side authentication and minimal local data. — Power controls: Limited scheduling and battery APIs; minimise radio and screen use. — Offline storage: Small record queue if persistent storage exists; avoid sensitive fields where encryption is absent. — Connectivity: Feature-phone cellular data/SMS or WML gateway depending on device.

Information retained for safe synchronisation

- Unique record ID generated on the device to make retries idempotent.
- Device ID and worker/account ID to identify the source and owner.
- Creation timestamp and sequence number to preserve order and support conflict resolution.
- Record version and schema version to handle updates and format changes.
- Sync status (pending, in progress, acknowledged, failed) to resume interrupted work.
- Cryptographic integrity/authentication metadata to detect tampering.
- Last successful sync cursor/time to request only outstanding records.

Stage 4: Complete solution specification

Architecture by device class

```
Presentation Layer
  Android UI / iOS UI / J2ME screens or WML pages
          |
Domain Layer
  Validation, patient-record rules, ID generation, sync policy
          |
Data Layer
  Encrypted local queue (A/B); minimal persistent queue (C)
          |
Platform Services
  Keystore/Keychain, WorkManager/BackgroundTasks,
  network APIs, secure transport, device storage
          |
Backend
  Authentication -> API Gateway -> Idempotent Sync Service
              -> Validation -> Patient Record Database
```

Data capture and offline storage

- Validate each field before saving; reject incomplete mandatory fields with a clear message.
- Generate a globally unique record ID before writing the record.

- Encrypt the record before placing it in the local queue on Android and iOS.
- Commit the record and its pending-sync status in one local transaction.
- For legacy devices, store only the minimum data supported by the device's security capability; if safe persistence is not available, use a short-lived session and server-side capture when connected.

Security and lost-device response

- Use authenticated user sessions and least-privilege access.
- Encrypt data at rest on modern devices using platform-backed key storage.
- Use HTTPS/TLS for every server transfer and validate certificates.
- Expire/revoke device credentials when a device is reported lost.
- Use remote lock/wipe or enterprise management where enrolled and supported.
- On Class C, minimise local patient details and use server-side access controls; acknowledge weaker at-rest protection as a residual risk.

Ten-hour power policy

- Keep the screen off when not actively entering data.
- Batch non-urgent uploads and run them when network and battery conditions are suitable.
- Use OS-managed background work rather than continuous polling.
- Sample location only at the frequency required by the field workflow.
- Avoid permanent wake locks; use short bounded wake periods for active capture or sync.
- Display a low-battery warning and preserve unsynchronised records.

Synchronisation procedure

21. Authenticate the worker/device and establish a TLS connection.
22. Select pending records in creation order and create a batch with a stable batch ID.
23. Send records with unique IDs and version metadata.
24. Server validates each record and uses the unique ID as an idempotency key; repeated uploads do not create duplicates.
25. Server returns per-record acknowledgements and the latest sync cursor.
26. Mark only acknowledged records as synced; keep unacknowledged records pending.
27. If connection fails, retain the queue and retry with bounded exponential backoff when connectivity returns.
28. If the device restarts during sync, resume from the persisted per-record status and resend safely.

Failure handling

Failure	Handling
No network	Keep encrypted queue locally and retry later.
App/process termination	Persist queue and sync state before background work; resume from durable state.
Interrupted upload	Resend unacknowledged records using idempotent record IDs.
Duplicate submission	Server uniqueness constraint/idempotency key prevents duplicate insertion.
Invalid or conflicting record	Return a per-record error; retain local copy for correction and audit.
Device lost	Revoke credentials, remote lock/wipe where supported, and report residual risk for legacy devices.
Storage full	Stop accepting new records safely, alert worker and preserve existing queued records.

Stage 5: Desk-check – Class A offline for six hours, 20% battery, then lost

29. At shift start, the worker authenticates and the app creates an encrypted local queue.
30. During six offline hours, records are validated, assigned unique IDs and stored encrypted with pending status.
31. Battery reaches 20%; the policy reduces non-urgent sampling and defers upload because there is no network.
32. The phone is lost before synchronisation; the server has no copy of the six-hour offline records.
33. The organisation reports the loss and revokes the device credentials; remote lock/wipe is attempted if the device reconnects.
34. Data exposed: ideally no plaintext patient records because local records and keys are protected by the OS and device lock; metadata may remain exposed depending on configuration.
35. Data lost: records not backed up to the server may be unrecoverable if the device cannot be recovered and no secure backup exists.
36. Data recoverable: previously synchronised server records remain available; unsynchronised local records are recoverable only if the device is recovered or a protected backup exists.

Stage 6: Adverse conditions and refinement

• Stolen unlocked phone — Prediction and result: Attacker may access data through the active app; encryption at rest alone does not prevent use of an unlocked session. — Refinement: Require re-authentication for sensitive actions, short auto-lock and remote credential revocation.

• Sync interrupted mid-transfer — Prediction and result: Some records may be acknowledged while others remain pending; duplicates are possible without idempotency. — Refinement: Persist per-record acknowledgements and use server-side unique IDs/idempotency.

• Class C has no secure storage — Prediction and result: Local patient data may be exposed if the feature phone is lost. — Refinement: Store only minimal/non-identifying data locally or require online-only access for sensitive records.

The most critical design decisions are unique record identifiers, durable local queue state, encryption on modern devices, server-side idempotency and minimisation of sensitive data on legacy devices. The Class C assumption may fail if the device cannot protect local records; the refined design therefore limits offline sensitive-data capability on that class.

Part I conclusion: A successful platform design begins with requirements and constraints, then assigns responsibilities to platform layers. The proposed RuralCare solution uses native platform security and storage where available, a constrained legacy client, durable offline queues and idempotent synchronisation.

PART II – MOBILE SYSTEMS ENGINEERING: IMPLEMENT, INSTRUMENT, TEST AND OPTIMISE

Part II is an implementation assignment. The following sections provide the design, test procedure, pseudocode and report structure. Actual screenshots, command outputs, logs, profiler traces and measurements must be collected on the student's own emulator/device. Do not submit expected values as measured results.

TASK 1 – ANDROID VS iOS RESOURCE-MANAGEMENT MODEL

1.1 Layer mapping

• Linux Kernel — Responsibility: Scheduling, memory, drivers, networking, security support. — iOS layer: Core OS — Responsibility: Kernel, drivers, networking, security and low-level services.

• HAL — Responsibility: Standard hardware interfaces for vendor implementations. — iOS layer: Core Services — Responsibility: Foundation, data, location, networking and core system services.

• Native libraries — Responsibility: Graphics, media, database, crypto and other native components. — iOS layer: Media — Responsibility: Graphics, audio, video and media processing.

• ART — Responsibility: DEX execution, JIT/AOT, garbage collection. — iOS layer: Cocoa Touch — Responsibility: UIKit, app lifecycle, touch input and app-facing frameworks.

• Application Framework — Responsibility: Activity, Service, BroadcastReceiver and system services. — iOS layer: UIKit/UIApplication — Responsibility: UIViewController lifecycle, app/scene states and UI management.

Process model: Android Zygote is the preloaded process that forks application processes; Activity Manager/ActivityTaskManager coordinates app launch. iOS launchd manages process launch and system services. IPC: Android uses Binder transactions for framework services; iOS uses Mach ports and XPC for process/service communication.

1.2 Evidence commands and interpretation

```
adb shell ps -A
adb shell dumpsys meminfo <package.name>
adb shell dumpsys activity processes
adb shell dumpsys package <package.name>
adb logcat
```

Capture the exact command output and replace [INSERT SCREENSHOT/LOG]. In process output, PID identifies the process; UID identifies its app security identity. In dumpsys meminfo, PSS estimates the process's proportional share of physical memory. Process importance/oom adjustment values indicate how strongly the OS should preserve or reclaim a process; lower-priority cached processes are more likely to be killed under memory pressure.

1.3 Three memory/foreground conditions

• Foreground — Prediction before test: Process should remain alive; foreground lifecycle state should be active. — Evidence to record: PID, process importance, lifecycle logs and saved-state output.

• Background for five minutes — Prediction before test: Process may remain cached or be reclaimed depending on memory pressure and OS policy. — Evidence to record: PID before/after, onStop/onSaveInstanceState logs, process state.

• Low-memory trim — Prediction before test: System may send trim callbacks and may kill a low-priority process. — Evidence to record: Trim callback, process list, lifecycle log and state restoration.

Use a test app that logs lifecycle callbacks and persists a small state value. Example test command for a controlled trim experiment: adb shell am send-trim-memory <package.name> RUNNING_LOW. Record the command and actual response. The exact lifecycle callback sequence depends on the app state and OS version; do not claim a callback occurred unless it appears in logs.

1.4 Boundary test

Rotate the screen while a form contains unsaved text, then force process death and reopen the activity. This boundary test checks whether transient UI state is saved independently of the process. Expected result: the form restores the explicitly persisted state; state held only in an Activity field may be lost.

TASK 2 – ARCHITECTURE ASSUMPTIONS VS MEASURED REALITY

2.1 Two implementation designs

• Compute-heavy work runs on the main/UI thread. — Solution B – background executor: Compute work runs on a background executor or coroutine.

• Easy to implement but can block rendering and input. — Solution B – background executor: UI thread remains available to render and respond.

• Use for tiny tasks only. — Solution B – background executor: Use for longer work; return results to UI thread safely.

Workload: parse a large route-data file or apply a repeatable image filter. Keep input data, algorithm and iteration count identical between A and B. Record elapsed time, CPU time, thread count, frame timing, memory/PSS and context switches. A multi-threaded version should not update UI widgets directly from worker threads.

2.2 Counter-example where more threads are worse

Use a very small operation repeated a limited number of times. In Solution B, creating workers, submitting tasks, synchronising and merging results can cost more than the computation itself. Prediction: multi-threaded elapsed time and CPU overhead may be higher than the single-threaded version. The first degrading decision is splitting a task too small to amortise scheduling and synchronisation overhead.

2.3 Measurement record

• Elapsed time (ms) — Single-threaded result: [MEASURE] — Multi-threaded result: [MEASURE]

• CPU time (ms) — Single-threaded result: [MEASURE] — Multi-threaded result: [MEASURE]

• Thread count — Single-threaded result: [MEASURE] — Multi-threaded result: [MEASURE]

• Context switches — Single-threaded result: [MEASURE] — Multi-threaded result: [MEASURE]

• Heap/PSS — Single-threaded result: [MEASURE] — Multi-threaded result: [MEASURE]

• Dropped/jank frames — Single-threaded result: [MEASURE] — Multi-threaded result: [MEASURE]

• Worst-case latency — Single-threaded result: [MEASURE] — Multi-threaded result: [MEASURE]

Use Android Studio Profiler, Perfetto or top -H to observe CPU/thread behaviour. For a battery-first working-day requirement, move non-urgent work to efficiency cores where the OS scheduler permits, batch work, avoid unnecessary thread creation and use WorkManager for deferrable work. The conclusion must be based on the collected device trace.

TASK 3 – RUNTIME BEHAVIOUR UNDER CHANGING CONDITIONS

3.1 Benchmark workloads

- CPU-bound workload: repeat a deterministic numeric algorithm with a fixed input size.
- Allocation-heavy workload: create and release many short-lived objects in a controlled loop.

- UI-list workload: render a long list with repeated rows and controlled image/text complexity.

3.2 Stage A – ART evidence

```
adb shell dumpsys package <package.name>
adb shell cmd package compile -m speed-profile -f <package.name>
adb logcat | grep -i -E "art|gc|concurrent copying"
adb shell dumpsys meminfo <package.name>
```

Record cold-start and warm-start time, compilation state, GC event count/pause duration and heap growth. Cold start means the process is not already resident; warm start means the process or relevant runtime state remains available. Use the same device, build, input and test conditions for each run.

3.3 Stage B – compilation mode change

• verify / interpret-oriented — Prediction: Lower compilation work/storage; repeated hot execution may be slower. — Workload likely affected: CPU-bound loop.

• speed-profile — Prediction: Frequently used code is compiled using profile information; startup and hot-path performance may improve. — Workload likely affected: Common launch and frequently executed CPU paths.

• speed — Prediction: More code compiled for performance; may increase install/compile time and storage. — Workload likely affected: Long-running CPU-bound workload.

Example command syntax must be checked against the installed Android version: adb shell cmd package compile -m verify -f <package.name>; adb shell cmd package compile -m speed-profile -f <package.name>; adb shell cmd package compile -m speed -f <package.name>. Capture the command output and compare actual results. Baseline Profiles can guide compilation of important startup and interaction paths.

3.4 Stage C – production failures

Memory leak test: retain Activity instances in a static collection or create an unbounded cache in a controlled test build. Detect increasing heap usage across repeated navigation, retained objects in a heap dump, GC thrashing or OutOfMemoryError. Main-thread blocking test: add a deliberately slow operation on the UI thread in a test build and observe StrictMode/ANR evidence. Remove the defects after collecting evidence.

3.5 Stage D – cross-platform runtime comparison

• ART + Kotlin/Java — Execution and memory: DEX, JIT/AOT and garbage collection. — UI/native access: Android framework and JNI/NDK. — Likely trade-off: Strong Android integration; GC and runtime heap require measurement.

• Swift + ARC — Execution and memory: Native compiled code; reference counting. — UI/native access: Direct Apple frameworks and native APIs. — Likely trade-off: Native platform integration; ARC ownership cycles must be managed.

• Flutter — Execution and memory: Dart AOT in release builds; garbage collection; own rendering engine. — UI/native access: Plugins/platform channels connect to native APIs. — Likely trade-off: Consistent UI across platforms; engine and framework add footprint.

• React Native / WebView hybrid — Execution and memory: JavaScript engine; bridge or JSI/native modules depending on architecture; garbage collection. — UI/native access: Native modules or Web APIs. — Likely trade-off: Code reuse; bridge/JS runtime and rendering path can affect startup and responsiveness.

A recommendation should be based on the same measured workloads. For a product requiring one codebase and a consistent interface, Flutter is a candidate when measured startup, memory and native plugin

performance meet targets. If deep platform integration and smallest platform-specific overhead dominate, native ART and Swift may be preferable. Do not claim a universal winner without evidence.

TASK 4 – SECURE MOBILE APPLICATION AUDIT

4.1 Audit input and threat model

Audit inputs: requested manifest/Info.plist permissions, storage locations, network endpoints/protocols and exported components/URL schemes. Use STRIDE categories: Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service and Elevation of Privilege.

- Auth token – information disclosure — Attack vector: Read local token file or backup. — Protection: Private app storage + Keystore-backed encryption. — Residual risk: Compromised/rooted OS or unlocked device may expose use of the token.
- Patient database – tampering — Attack vector: Modify local SQLite/file data. — Protection: Encrypted database, integrity checks and access controls. — Residual risk: Valid app process or compromised device may alter data.
- API traffic – disclosure/tampering — Attack vector: Intercept network traffic. — Protection: TLS with certificate validation; pinning where operationally appropriate. — Residual risk: Compromised endpoint/device or certificate/key-management failure.
- Exported activity – spoofing/privilege abuse — Attack vector: External app invokes an exposed component. — Protection: Set exported=false unless required; validate caller/input and permissions. — Residual risk: Misconfigured deep links or exported entry points remain risks.
- Camera/location – privacy misuse — Attack vector: Abuse overbroad permissions. — Protection: Runtime permissions and purpose-specific access. — Residual risk: User-granted permission can still be misused by app code.
- Malicious package update – tampering — Attack vector: Install altered package/update. — Protection: App signing, trusted distribution and verified update identity. — Residual risk: Signing-key compromise or supply-chain vulnerability.

4.2 Security decision trace

Asset	Attack vector	Mechanism applied	Result / reason
Auth token	Other app reads token file.	Private storage + Keystore encryption.	Blocked under normal sandbox conditions; rooted device residual risk.
Local DB	Offline file theft.	Encrypted database and key protection.	Partially blocked; unlocked/compromised OS remains risk.
API traffic	Network interception.	TLS certificate validation.	Blocked for passive interception; endpoint compromise remains.
Exported activity	Untrusted app launches component.	Not exported unless required; input validation.	Blocked if manifest and runtime checks are correct.

4.3 Weaken one control – prediction

Selected control: allow cleartext traffic. Prediction: API traffic becomes exposed to passive network observers and may be modified by an active attacker. The audit verdict should change from protected to

exposed/partially blocked. Verify only against a test application and controlled test server; capture the network trace and compare it with the original TLS configuration.

4.4 Third-party library leaks device identifier

Affected asset: device identifier and associated user/device metadata. Affected layer: application/library and transport layer. The sandbox does not prevent a library running inside the authorised app process from sending data through the app's network permission. Detect using static code/dependency review, network capture and runtime instrumentation. Remediation: remove or update the library, minimise identifier access, enforce TLS, restrict outbound endpoints and audit data collection. TLS protects transport but does not make unauthorised collection legitimate.

4.5 Biometric and BYOD extension

Biometric login adds a trust boundary between the user, biometric subsystem and app. Bind cryptographic key use to device authentication where supported; do not store biometric templates in the app. BYOD adds a separation boundary between personal and work data. Update the threat model for cross-profile data leakage, work-profile backup, clipboard/sharing, device compliance and remote selective wipe. Audit logic must inspect key attestation where available, profile ownership, policy state and whether work data can be selectively removed.

TASK 5 – POWER MANAGEMENT: DVFS, DOZE AND APPLICATION CONTROL

5.1 Three techniques

- DVFS and core selection — Trigger / maintained state: CPU utilisation, thermal state, workload and scheduler policy. — Power effect: Lower frequency/voltage or use efficiency cores can reduce CPU energy. — Side effect / evidence: May increase task completion time; capture frequency/core and CPU usage trace.
- Doze / App Standby — Trigger / maintained state: Screen off, stationary/idle state, app usage bucket and system constraints. — Power effect: Defers jobs/network and reduces repeated wake-ups. — Side effect / evidence: Delayed background work; capture batterystats/Doze state.
- Wakelock discipline and batching — Trigger / maintained state: Explicit short task requirement, job constraints and queued work. — Power effect: Reduces unnecessary CPU/radio wake periods and coalesces network transfers. — Side effect / evidence: Updates may be delayed; capture wake-lock duration, radio-active time and job count.

5.2 Conflict stress test

Create a test case that holds a partial wake lock while the device is otherwise idle. Prediction: the wake lock can keep the CPU available and reduce the benefit of Doze. A second test can schedule frequent exact alarms that repeatedly wake the device and defeat batching. Record the system state, wake-up count and wake-lock duration before and after the change.

5.3 Deployment decision

- A – Always-on tracking — Reasoned strategy: Use carefully bounded foreground tracking, sensor batching and adaptive sampling; continuous location may require higher power and must be justified.
- B – Media/gaming — Reasoned strategy: Use DVFS and scheduler/core selection for sustained CPU/GPU demand; optimise rendering and frame rate rather than aggressive background batching.

• C – Managed fleet — Reasoned strategy: Use OS-level job scheduling, policy-controlled background work and version-aware management; test across OS versions because restrictions differ.

A technique at one layer can be defeated by another layer: a permanent application wake lock can prevent system idle savings, while repeated alarms can undo network batching. The system must be measured as a whole rather than evaluating each feature in isolation.

TASK 6 – BATTERY-AWARE, SECURE FIELD-TRACKING PROTOTYPE

6.1 Problem model

• Input — Definition: Battery percentage, charging state, network type, motion state, temperature, last upload time and queued record count.

• Output — Definition: Sampling interval, upload decision, encryption status and estimated energy per cycle.

• Objective — Definition: Maximise useful tracking records delivered per unit of battery energy while protecting data.

• Constraints — Definition: Shift duration, battery, network availability, emergency latency, privacy and thermal limits.

Battery percentage alone is insufficient. Two devices at 20% may need different actions: one may be stationary on Wi-Fi and not charging, while another may be moving with an urgent delivery event and LTE connectivity. Charging state, motion, network, queued data, last upload and temperature distinguish these cases.

6.2 State-based policy

• High (>50%) — Network state: Wi-Fi — Motion state: Moving — Sampling policy: Every 10 s — Upload action: Batch upload every 2 min.

• High (>50%) — Network state: LTE — Motion state: Moving — Sampling policy: Every 15 s — Upload action: Upload small batches every 5 min.

• Medium (20–50%) — Network state: Wi-Fi — Motion state: Stationary — Sampling policy: Every 60 s — Upload action: Upload pending batch.

• Low (≤20%) — Network state: LTE — Motion state: Stationary — Sampling policy: Every 120 s — Upload action: Defer non-urgent upload; retain encrypted queue.

• Any — Network state: No connectivity — Motion state: Moving — Sampling policy: Every 30 s or policy limit — Upload action: Queue encrypted records; retry later.

• Any — Network state: Charging + Wi-Fi — Motion state: Any — Sampling policy: Normal required rate — Upload action: Upload pending batch promptly.

• Hot device — Network state: Any — Motion state: Any — Sampling policy: Reduce non-critical sampling — Upload action: Defer non-urgent processing until temperature normalises.

Default state: battery state unknown until read; network unknown; motion stationary until sensor evidence updates it. The policy begins in a conservative sampling mode, encrypts records before queueing and uses authenticated TLS uploads for every network transition. State changes occur when battery thresholds, charging state, network type, motion classification or thermal threshold changes.

6.3 Layered architecture

```
UI (Main Thread)
  Status, consent, battery and sync display
          |
Domain Layer (Background Executor)
  Policy Engine -> Sampling Scheduler -> Record Validator
          |
Data Layer
  AES-GCM encrypted queue + sync cursor + local metadata
          |
OS Services
  Location/Sensors | WorkManager | Foreground Service
  Keystore | Connectivity | Battery/Thermal APIs
          |
Network
  TLS API -> Authentication -> Idempotent Server Ingestion
```

6.4 Prototype policy logic (language-independent pseudocode)

```
ALGORITHM EvaluateTrackingPolicy(Battery, Charging, Network, Motion, Temperature)

INPUT:
   Battery, Charging, Network, Motion, Temperature
OUTPUT:
   SamplingInterval, UploadAction

BEGIN
   IF Temperature = HOT THEN
      SamplingInterval <- 120 seconds
      UploadAction <- DEFER_NON_URGENT
   ELSE IF Charging = TRUE AND Network = WIFI THEN
      SamplingInterval <- REQUIRED_NORMAL_RATE
      UploadAction <- UPLOAD_PENDING_BATCH
   ELSE IF Network = NONE THEN
      SamplingInterval <- 30 seconds
      UploadAction <- STORE_ENCRYPTED
   ELSE IF Battery <= 20 AND Motion = STATIONARY THEN
      SamplingInterval <- 120 seconds
      UploadAction <- DEFER_TO_WIFI_OR_CHARGING
   ELSE IF Battery <= 50 THEN
      SamplingInterval <- 60 seconds
      UploadAction <- BATCH_UPLOAD
   ELSE
      SamplingInterval <- 10 seconds
      UploadAction <- BATCH_UPLOAD
   END IF

   Encrypt record before local queue write
   Authenticate and use TLS before upload
   RETURN SamplingInterval, UploadAction
END
```

6.5 Six-condition scenario set

• Charging + Wi-Fi — Policy output: Normal sampling; upload pending batch. — Security / energy record: Encrypt before storage; TLS upload; estimate mAh from measured profile.

• 80% battery + LTE + moving — Policy output: 15 s sampling; small periodic batch. — Security / energy record: Encrypted queue; TLS; record radio-active time.

• 20% battery + stationary + LTE — Policy output: 120 s sampling; defer non-urgent upload. — Security / energy record: Encrypted queue; record wake-ups and estimated mAh.

• Low-power mode — Policy output: Reduce non-critical sampling; defer jobs. — Security / energy record: Encrypted queue; record OS policy state.

• No connectivity — Policy output: Continue bounded sampling; queue records. — Security / energy record: Encrypted queue; no radio retry loop.

• Device hot — Policy output: Reduce sampling and defer non-urgent work. — Security / energy record: Encrypted queue; record thermal state and CPU activity.

Energy estimates must be calculated from measured device data or a clearly identified documented power model. Fill in: [INSERT measured energy per cycle], [INSERT sampling interval observed], [INSERT upload result] and screenshots/log references.

6.6 Validation

For three test conditions, calculate expected policy outputs independently by hand and compare them with prototype output. Example: (1) 80% battery, LTE, moving → 15-second sampling and small batch upload; (2) 20% battery, stationary, LTE → 120-second sampling and deferred upload; (3) no network → encrypted queue and no repeated radio retry. Confidence increases when independently calculated outputs match the prototype for normal, boundary and failure cases.

6.7 Scalability/load test

• 100 records — Policy evaluations: [MEASURE] — Wake-ups: [MEASURE] — Memory / battery observation: [MEASURE]

• 1,000 records — Policy evaluations: [MEASURE] — Wake-ups: [MEASURE] — Memory / battery observation: [MEASURE]

• 10,000 records — Policy evaluations: [MEASURE] — Wake-ups: [MEASURE] — Memory / battery observation: [MEASURE]

• 1 / 5 / 10 / 20 devices — Policy evaluations: [MEASURE] — Wake-ups: [MEASURE] — Memory / battery observation: [MEASURE]

Batching reduces repeated radio wake-ups because several records share one radio activation and connection setup. It does not make continuous high-frequency tracking cheap: sensors, CPU processing, GPS and the volume of collected data still consume energy, and large batches may increase memory/storage use.

6.8 Business requirement change: 5,000 devices and emergency updates

The existing battery-saving policy becomes unsuitable because its deferred sampling and upload windows may violate the new few-seconds emergency latency requirement. The changed property is the priority of the objective: reliable, timely delivery now outweighs battery conservation during emergencies. Investigate an adaptive priority policy that raises sampling frequency during emergency state, uses an eligible foreground service, high-priority push/transport where supported, edge/cloud offloading and server-side fleet coordination. The accepted trade-off is higher energy use, heat, network cost and potentially greater privacy exposure from more frequent location transmission.

6.9 Technical reflection (under 150 words)

No single power-management feature is optimal because applications have different timing, workload and reliability requirements. In a stationary low-battery condition, slower sampling and batched uploads reduce wake-ups and radio activity. During an emergency, the same delay would make location information less useful. A wake lock may be necessary for a short critical operation but harmful if held continuously because it

prevents the device from sleeping. DVFS can reduce energy during light work but may increase completion time under heavy processing. Therefore, the policy must consider battery, charging, network, motion, temperature and urgency together. The final choice should be supported by measurements of wake-ups, CPU use, radio-active time, memory and delivery delay.

ADDITIONAL ENGINEERING EXPLANATION AND TEST GUIDANCE

1. Reproducible Testing Procedure

A fair mobile-system experiment must control the conditions that can change the result. Use the same device or emulator profile, Android version, application build, input dataset, network condition and test duration when comparing two implementations. Record the date, device model, OS build, available memory and battery state before each run.

Run each benchmark more than once. A single measurement can be affected by background updates, thermal throttling, compilation state or network variation. Report the individual observations and a representative average or median. If one run is unusually different, explain the likely cause rather than silently removing it.

Separate prediction from observation. Before changing a compilation filter, memory condition, security control or power policy, write what you expect to happen and why. After the test, compare the actual evidence with the prediction. A mismatch is useful engineering evidence because it may reveal an incorrect assumption about the OS or workload.

Keep a small experiment record for every test: purpose, configuration, input, steps, expected result, actual result, screenshot or log filename, and interpretation. This makes the work repeatable and allows another engineer to follow the same procedure.

2. Interpreting Measurements Correctly

Elapsed time alone does not describe mobile performance. A background implementation may finish sooner but consume more CPU time, create more threads, allocate more memory or keep the radio active for longer. Compare computational cost, memory footprint, responsiveness and worst-case behaviour together.

For memory experiments, distinguish managed heap size from total process memory. PSS estimates the process's proportional share of physical memory and is useful for comparing memory pressure. A process disappearing from the process list does not by itself prove an application defect; Android may reclaim cached processes to protect foreground work.

For power experiments, battery percentage is a coarse indicator and may change slowly. Wake-up count, partial wake-lock duration, radio-active time, CPU frequency, thermal state and job scheduling behaviour help explain why energy use changed. Use the same observation interval for every comparison.

For security experiments, evaluate each control against a specific threat. TLS protects data in transit but does not prevent an authorised application component from collecting data unnecessarily. A sandbox limits access between applications but cannot fully protect data from malicious code already running inside the authorised application process.

3. Design Boundaries and Failure Recovery

The offline field-data design depends on durable local state. A record should be committed to the local queue before the application reports that it has been saved. The record identifier and sync status must also be persisted so that a restart does not create a new identity for the same record.

A network request can fail after the server has stored a record but before the device receives the acknowledgement. The client cannot safely assume that the record was rejected. It should retry using the same

unique record ID, while the server treats repeated requests as the same logical operation. This is the purpose of idempotent synchronisation.

Security and reliability must be considered together. Encryption protects a stored record from casual file access, while key protection controls access to the decryption key. Remote wipe is useful only when the device reconnects and receives the command; therefore, the design should also revoke credentials on the server so a lost device cannot continue using an old session.

Legacy feature phones may not provide the same sandbox, encryption and remote-management facilities as modern smartphones. The correct response is to reduce the amount and sensitivity of information stored locally, not to claim that the older device offers equivalent protection.

4. Submission Quality and Result Interpretation

Every screenshot should show enough context to identify the test being performed. Include the command or application screen, the relevant output, and a short caption explaining what the evidence proves. Avoid screenshots that show only a final percentage or a successful build message.

When reporting a result, distinguish measured values from estimates. A value obtained from a profiler is a measurement; a value calculated from a documented power model is an estimate. State the method and units so that the reader can understand the strength and limitation of the evidence.

The final discussion should answer three questions: Did the implementation meet the requirement? Under what condition did it become less effective or fail? What design change would improve it without creating a larger problem elsewhere? This demonstrates engineering reasoning rather than merely listing features.

PART II – EVIDENCE CHECKLIST

- Add the Android device/emulator model, OS version, RAM setting and test-app package name.
- Attach source code for each implemented test and prototype.
- For each test, show input, predicted behaviour, actual output and interpretation.
- Include screenshots of adb commands, logs, profiler traces and relevant system state.
- Record actual cold/warm start, memory, GC, CPU, thread, frame, wake-lock, radio and energy observations where required.
- Show at least one boundary/failure test and explain why the result occurred.
- Replace all [MEASURE] and [INSERT] fields with genuine observations before submission.

OVERALL CONCLUSION

Assignment 4 examines mobile platforms as complete resource-management systems. Part I develops precise architecture, runtime, security, power and platform-strategy designs. Part II requires those ideas to be tested through implementation and instrumentation. The key engineering principle is to connect every design decision to observable behaviour: process lifecycle, memory, runtime compilation, security boundaries, power-state transitions, synchronisation and failure recovery. A design is not validated merely because an application launches; it must behave correctly under changed requirements and adverse conditions.

REFERENCES

37. SIMATS Engineering, ITA03-MP Assignment 4A: Mobile Application Development Environment – Assignment 4, Part I, supplied course handout.
38. SIMATS Engineering, ITA03-MP Assignment 4B: Mobile Computing – Assignment 4, Part II, supplied course handout.
39. Android Developers documentation: Application fundamentals, processes and threads, app permissions, background tasks, Doze and App Standby, Android Runtime and app signing.

40. Apple Developer Documentation: App lifecycle, background execution, App Transport Security, Keychain Services and App Sandbox.
41. Microsoft Learn: Windows app lifecycle, WinRT, AppContainer and application security.